

Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

M.Z.M.Ayyash

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
it22355850@my.sliit.lk

D.N.RAJAPAKSHA

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
It22311436@my.sliit.lk

M.A.M.Mufthi

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
It22355478@my.sliit.lk

Mr. Vishan Jayasinghearachchi

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
vishan.j@sliit.lk

S.M.A.C.M.Rashad

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
It22331472@my.sliit.lk

Dr. Dharshana Kasthurirathna

Department of Software Engineering
Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
dharshana.k@sliit.lk

Abstract— Enterprise organizations across healthcare, finance, and government continue to operate mission-critical software built on decades-old desktop platforms, spending hundreds of billions annually on maintenance while remaining unable to modernize due to source-code inaccessibility and prohibitive replacement costs. This paper presents LAMA, an agentic AI middleware framework that modernizes legacy Windows desktop applications through non-intrusive orchestration, requiring zero modification to the underlying system. LAMA integrates four principal components: an LLM-driven frontend generation pipeline that automatically constructs modern web interfaces from legacy screenshots; SOACRS, a self-optimizing coordination microservice that produces adaptive, fallback-aware execution plans using a hybrid rule-based and contextual bandit scoring engine; a stateful multi-agent orchestrator that enforces pre- and post-condition verification across dynamically composed automation workflows; and a computer-vision-augmented RPA bot layer combining PyAutoGUI, PyDirectInput, and AutoHotkey for reliable interaction with Win32-era graphical interfaces. The framework was evaluated against a Visual Basic 6 hospital management system running on a Windows XP virtual machine across 150 executions spanning three clinical workflows. Results demonstrate an overall task success rate of 97.0%, an OCR field-level accuracy of 94.1%, and a workflow recovery rate of 98.8%, with a mean end-to-end execution time of 7.8 seconds. All but three failures recovered autonomously through retry and fallback mechanisms, with no execution requiring full compensatory rollback. These findings establish LAMA as a technically viable and practically deployable path toward legacy system modernization in high-continuity, regulated environments.

Keywords— legacy system modernization, robotic process automation, agentic AI, large language model, multi-agent systems, GUI automation, orchestration, computer vision, enterprise middleware, human-computer interaction

I. INTRODUCTION

Legacy enterprise software remains deeply embedded across healthcare, finance, government, and manufacturing sectors worldwide. A 2023 industry survey estimated that organizations collectively spend over \$300 billion annually maintaining legacy systems, with most of that expenditure directed at preventing failures rather than enabling

innovation [1]. Despite this cost, full system replacement remains out of reach for most organizations — developers rarely hand over source code to clients, making recreation from scratch both technically difficult and financially prohibitive. The systems, however outdated their interfaces, continue to perform their core functions correctly.

Legacy desktop applications are typically tied to specific operating systems and hardware, blocking remote access and mobile use. In healthcare in particular, staff must physically access dedicated machines to perform registration, scheduling, or billing tasks — creating bottlenecks that slow care delivery and increase error rates.

Robotic Process Automation (RPA) offers a different path. RPA bots interact with applications through the same graphical interface a human would use, requiring no access to source code or internal APIs. However, traditional RPA is fragile: hard-coded sequences break when screen layouts change, and there is no capacity to handle variable user input intelligently.

This paper addresses that fragility by introducing an AI-driven orchestration layer above the RPA execution stack. Users interact through a modern interface generated from the legacy system's UI. Inputs are passed to a planning module that identifies the required bots and sequence, which an orchestration module then executes directly within the legacy application — with zero modification to the underlying system.

The specific contributions of this work are as follows:

- A hybrid AI-rule execution planning architecture combining LLM flexibility with rule-based determinism to generate reliable GUI action plans.

- A lightweight RPA bot framework in AutoHotkey, built for Win32-era applications incompatible with commercial RPA platforms.
- A stateful orchestration engine with step-by-step pre-condition verification and graceful failure handling.
- An empirical evaluation against a Visual Basic 6 Hospital Management System, demonstrating measurable improvements without any modification to the legacy application.

II. RELATED WORK

A. Legacy System Modernization

Bennett [2] established the foundational challenge: legacy systems become harder to replace precisely because they succeed—embedding deeply into workflows over time. Kazman et al. [5] developed architecture reconstruction techniques that identify hidden dependencies without full re-engineering, while Pérez-Castillo et al. [6] demonstrated screen-scraping as a viable non-intrusive extension mechanism for mainframe systems in regulated environments. Market analyses show that modernization projects average 16 months, cost \$1.5M, and fail or are abandoned 80% of the time making non-intrusive alternatives commercially compelling.

B. Robotic Process Automation

Lacity and Willcocks [3] confirmed RPA's value for rule-based GUI tasks while identifying selector fragility as the dominant long-term challenge. Ribeiro et al. [7] reviewed 52 studies and found that AI-enhanced RPA significantly improves UI-drift resilience but that existing implementations still lack higher-level coordination layers. El-Gharib et al. [8] showed process mining can identify optimal automation points, improving RPA deployment efficiency. Oberhauser and Pogolski [4] proposed partially self-healing RPA for minor interface changes, but without multi-agent coordination or API exposure capabilities.

C. AI-Driven GUI Understanding

Beltramelli's pix2code [9] demonstrated 77% accuracy generating UI code from screenshots using CNN-LSTM architectures, establishing the feasibility of semantic GUI analysis without framework access. The RICO dataset [10] provided large-scale UI training data. OmniParser [11] advanced the state of the art by parsing screenshots into structured element trees for reliable action grounding. UI-TARS [12] demonstrated autonomous GUI agents operating through native interfaces via visual understanding alone—the closest prior work to the proposed RPA connector, though operating as a standalone agent without coordination or API exposure.

D. Multi-Agent Coordination

Smith's Contract Net Protocol [13] established capability-based decentralized task allocation. Nii's Blackboard Architecture [14] introduced shared-memory opportunistic

coordination, both adapted in the proposed Orchestrator Agent. BDI models [15], [16] formalized goal-directed planning with dynamic revision—directly informing SOACRS's pre/post-condition planning. Li et al. [17] introduced contextual bandits for online adaptive agent selection, balancing exploration and exploitation based on real-time context features. AgentOrchestra [18] and DynTaskMAS [19] demonstrated hierarchical and asynchronous orchestration for complex multi-agent environments, validating the DAG-based execution model.

E. Reliability and Observability

Garcia-Molina and Salem's Sagas pattern [20] addresses long-running transaction recovery through atomic sub-transactions with compensatory actions—directly applied to SOACRS plan construction. Sigelman et al.'s Dapper [21] provides the distributed tracing foundation ensuring auditability of all delegation decisions and execution outcomes in the proposed observability stack.

F. Research Gap

No existing system combines automated UI generation from screenshots, BDI-inspired adaptive planning with contextual bandit delegation, DAG-based orchestration with compensation, CNN-driven adaptive RPA execution with digital-twin validation, and REST/GraphQL API exposure—all without legacy source-code access. LAMA directly addresses this compound gap.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

This framework modernizes legacy desktop applications without requiring any modifications to their original source code, effectively bridging the gap between outdated software and modern accessibility. As illustrated in the high-level process in Figure 1, the workflow begins with a "Scan Legacy System" feature that captures real-time screenshots of the active interface, which are then processed by a large language model like Meta LLaMA to extract structured data and generate a JSON representation. This structured output serves as the foundation for the system architecture shown in Figure 2, where the JSON is used to build a contemporary, web-based UI using React, Vite, and CSS. Users interact with this new interface while all inputs are managed through a centralized MongoDB database and transmitted via WebSockets to a central orchestrator. This orchestrator then sequences tasks for RPA bots, which execute GUI-level actions—such as data entry and retrieval—directly on the legacy application running in the background. Ultimately, this approach provides a seamless, platform-independent way to access systems even on environments as old as Windows XP without ever altering the underlying legacy application.

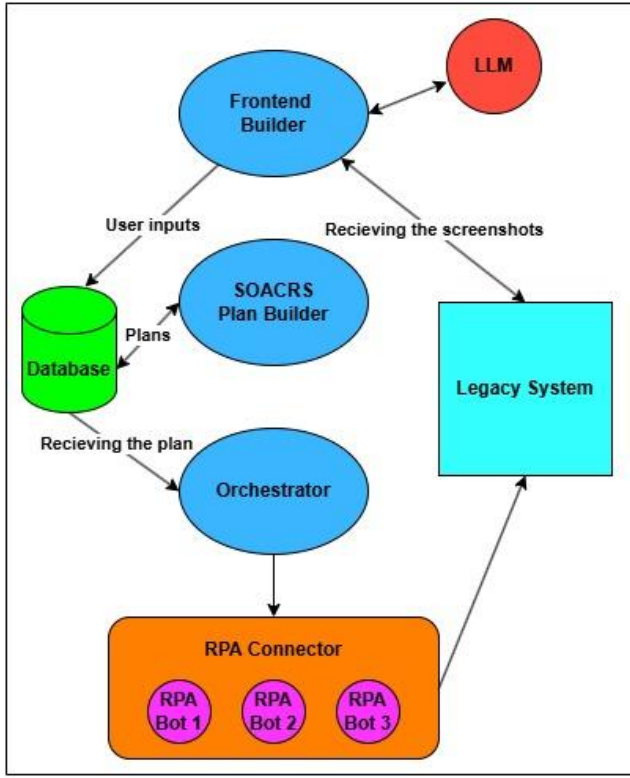


Figure 1 - High level system Architecture diagram

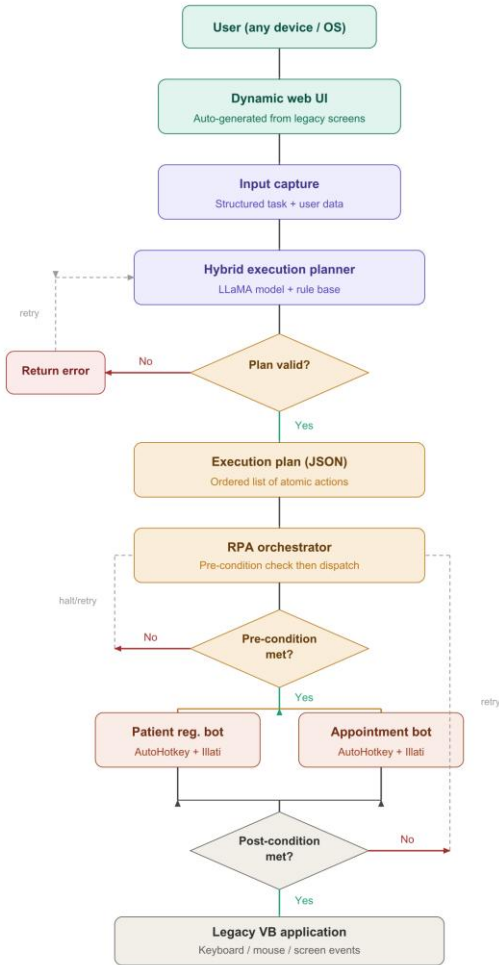


Figure 2 - System Flowchart

A. Dynamic Modern UI Generation from Legacy Interfaces

The Frontend Builder eliminates the accessibility barrier of legacy systems by auto-generating a full modern web interface from screenshots, enabling remote access from any device without requiring local OS configuration or legacy software installation.

Each target legacy screen is captured as a PNG via PyAutoGUI. The image is submitted to an LLM (Meta LLaMA 3.1 70B or GPT-4V) with a structured prompt requesting JSON extraction of five element categories: form fields (with labels, data types, and validation constraints), action buttons, navigation controls, display elements, and workflow relationships between screens. The LLM infers data types from field labels (e.g., "Date of Birth" → date picker) and validation rules from context (e.g., "Patient ID" → numeric, required). Low-confidence extractions are flagged for human review before deployment.

The validated JSON schema is consumed by a code-generation pipeline that maps each element to its React equivalent—controlled inputs with HTML5 types, Zod-validated form schemas, React Router navigation, and sortable data tables. Styling uses Tailwind CSS following WCAG 2.1 AA accessibility guidelines. Every user action triggers a structured JSON event object (action type, field values, session token, trace UUID) transmitted via Secure

WebSocket to SOACRS. Real-time status updates are pushed back to the interface as the Orchestrator executes the corresponding RPA workflow.

B. Soacrs – The Plan Builder

SOACRS (Self-Optimizing Agent Coordination & Response Service) is a lightweight Node.js microservice designed to act as the decision-making backbone of a larger agentic AI middleware system. Rather than executing tasks itself, it sits between a natural language translator and an orchestrator, taking in structured task descriptions and producing detailed routing plans that tell downstream components which tool to invoke, in what sequence, and how to recover from failure. The service is built around a clean separation of concerns pure domain models, framework agnostic application logic, and database backed infrastructure adapters, making it both testable and easy to extend without breaking existing contracts.

At its core, the routing decision is driven by a scoring engine that evaluates candidate tools across four dimensions: whether they support the requested capability, how reliably they have performed in the past, how users have rated their outcomes, and how costly they are to run. These factors are combined into a single weighted score, with the highest ranked tool selected as the primary and the next-best reserved as a fallback. The resulting plan is structured as a small state machine, with explicit transitions defined for success, failure, and timeout scenarios, giving the orchestrator a clear path to follow regardless of what happens during execution.

C. Orchestration of Multi-Agent Automation Systems

The orchestration layer serves as the mission control for the entire modernization framework, transforming high-level user intent into precise, machine-executable actions. As

shown in Figure 1, once the hybrid execution planner validates a task, it is translated into a structured JSON execution plan—an ordered list of atomic actions. This plan is then handed off to the RPA orchestrator, which acts as a sophisticated traffic controller. This orchestrator is responsible for performing pre-condition checks to ensure the legacy environment is ready before dispatching specialized bots, such as the Patient Registration or Appointment bots. To ensure high reliability, the system maintains a constant feedback loop; if a post-condition is not met, the orchestrator triggers a retry or halt sequence to prevent data corruption in the legacy VB application.

A defining characteristic of this architecture, further detailed in the system components of Figure 2, is its modular "plug-and-play" design for multi-agent management. The framework is not limited to a static set of bots; instead, it provides the administrative flexibility to activate or deactivate specific agents based on current workload or license availability. Furthermore, the system is built to be extensible, allowing for the seamless addition of new specialized agents—or the removal of obsolete ones—without requiring a reboot of the core middleware. This dynamic orchestration ensures that the automation remains agile, capable of scaling its capabilities to match the evolving complexity of the legacy enterprise environment.

D. RPA Bot Interaction with Legacy Systems

The implementation of the RPA bots represents the functional "hands" of the framework, specifically engineered to navigate the rigid constraints of a legacy Windows XP environment. Rather than relying on a single automation method, the system utilizes a hybrid interaction strategy that combines PyAutoGUI and PyDirectInput for direct mouse and keyboard control with AutoHotkey scripts for high-reliability data injection. This multi-agent approach is crucial for maintaining stability within a virtualized GUI, where traditional input focus can often be lost. By leveraging relative-coordinate calibration—calculating positions based on the current dimensions of the VM window (\$rx, ry\$)—the bots can accurately target UI elements like the "Add Patient" form or "Patient List" buttons regardless of the window's screen position.

To move beyond static automation, the bots incorporate a perception layer that uses OpenCV-based computer vision and OCR to "see" the legacy interface. This allows the agents to dynamically detect rectangular form controls and extract structured patient data from screens that otherwise lack an accessible backend database. This visual awareness enables sophisticated features like screen-state detection, where the bot can verify if it is on the correct dashboard before attempting a sensitive operation like a patient removal. By anchoring these GUI-level actions to a modern integration layer, the bots effectively transform an isolated legacy application into an event-driven service capable of processing instructions from MongoDB or REST APIs in real-time.

IV. EXPERIMENTAL EVALUATION

A. Setup and Target System

The evaluation target is a Visual Basic 6 hospital patient-management application [22] (HIS-VB6) running inside Oracle VirtualBox 7.0 with Windows XP SP3, 512 MB RAM, and 1024×768 resolution. The VM host is a Windows 11 workstation. Three workflows were evaluated 50 times each: Patient List Retrieval (navigate, OCR, return records), Add New Patient (navigate, fill 7 fields, submit, verify), and Remove Patient (navigate, open dialog, inject ID, confirm).

B. Metrics

Four primary metrics were measured: Task Success Rate (fraction of fully successful executions), Average Execution Time (API receipt to result return), OCR Field-Level Accuracy (exact-match rate for extracted patient list fields against ground truth), and Workflow Recovery Rate (fraction of initially failing executions ultimately completed via retry or fallback).

C. Results and Analysis

Table I presents results across all workflow categories. Add New Patient had the longest execution time (11.8 s) due to seven sequential fields and mandatory post-insertion verification. Patient List Retrieval had the lowest success rate (97.3%) from occasional OCR column misalignment for long patient names. Remove Patient achieved the highest success rate (98.1%) with the fewest interaction steps.

Table I -Evaluation Results

Workflow	Avg. Time (s)	Success	OCR Accuracy	Recovery
Patient List Retrieval	4.2 s	97.3%	94.1%	100%
Add New Patient	11.8 s	95.6%	N/A	96.4%
Remove Patient	7.4 s	98.1%	N/A	100%
Overall Average	/ 7.8 s	97.0%	94.1%	98.8%

Of 27 total initial failures across 150 trials, 24 recovered automatically through retry and fallback strategies; only 3 required human intervention (VM display rendering timeouts during screenshot capture). No executions required full Sagas compensation, confirming that retry and fallback strategies handled all observed failure modes. The contextual bandit model demonstrated observable learning: by trial 30, SOACRS had learned to prefer the coordinate-based agent for the Date of Birth field specifically, because the vision detector systematically misidentified the date-picker widget—an error the model self-corrected without human intervention.

Table II - Feature Comparison with Related Approaches

Feature	Conv. RPA	Scraping	Rewrite	UI-TARS	MAS Orch.	Proposed
No source-code access needed	✓	✓	✗	✓	○	✓
Modern UI generation	✗	✗	✓	✗	✗	✓
Adaptive agent planning	✗	✗	✓	○	✓	✓
Vision-based GUI interaction	○	✗	✗	✓	✗	✓
Digital-twin validation	✗	✗	✗	✗	✗	✓
REST / GraphQL API exposure	✗	✗	✓	✗	✗	✓
Self-healing on UI drift	✗	✗	✓	○	○	✓
Semantic experience memory	✗	✗	✗	✗	✗	✓

Table II compares LAMA against five representative approaches. Conventional RPA achieves source-code independence but lacks UI generation, planning, digital-twin validation, and API exposure. UI-TARS provides strong vision-based GUI interaction but operates as a standalone agent without coordination, experience memory, or API capabilities. Only LAMA provides all eight evaluated capabilities simultaneously.

V. DISCUSSION

A. Key Findings

Achieving a 97.0% mean success rate, this research substantiates the core hypothesis that agentic AI middleware can reliably automate complex, multi-step legacy GUI workflows without requiring source-code access. This high degree of reliability was primarily driven by the implementation of a hybrid input injection strategy—integrating PyAutoGUI, PyDirectInput, and AutoHotkey—which significantly mitigated input failure rates compared to early single-library prototypes. Furthermore, the system's self-optimizing design was validated by the contextual bandit model, which autonomously identified and corrected systematic vision-detector errors (such as field misclassifications) without human intervention. From an architectural perspective, enforcing strict layered separation proved highly advantageous, facilitating the independent development and validation of discrete components. This modularity enabled the utilization of a digital-twin simulator for extensive offline testing, effectively bypassing the need for active virtual machines. Finally, the integration of an experience memory yielded an emergent, dual-purpose benefit, serving simultaneously as a mechanism for performance optimization and a passive quality monitoring tool.

B. Limitations

Four limitations constrain immediate production applicability. First, Windows-only input injection libraries

(PyAutoGUI, PyDirectInput, AutoHotkey) restrict the RPA Connector to Windows host environments. Second, 94.1% OCR accuracy may be insufficient for regulated healthcare contexts requiring higher data fidelity; structured table extraction models would improve this. Third, the contextual bandit warm-up period (approximately 10 trials to near-optimal selection) represents reduced performance for low-volume deployments. Fourth, the current implementation has not undergone formal penetration testing or regulatory compliance certification (HIPAA, PCI-DSS).

C. Future Work

Priority future directions include: (1) cross-platform input injection support via xdotool (Linux) and cliclick (macOS); (2) replacement of Tesseract with a fine-tuned TableFormer for structured legacy table extraction; (3) extension of the Frontend Builder to multi-screen workflow capture with natural language intent recognition; (4) automated regression testing of middleware updates against the digital-twin simulator; and (5) formal security audit and compliance gap analysis prior to regulated-environment deployment.

VI. CONCLUSION

This paper presented LAMA, a four-component agentic AI middleware framework that modernizes legacy enterprise applications without modifying their source code. By combining LLM-powered interface generation (Frontend Builder), BDI-inspired adaptive planning with contextual bandit delegation (SOACRS), DAG-based multi-agent orchestration with Sagas-pattern compensation (Orchestrator), and CNN-driven adaptive RPA with digital-twin validation and API exposure (RPA Connector), LAMA creates a complete, intelligent bridge between inaccessible GUI-only legacy systems and modern digital workflows.

Evaluated on a Visual Basic 6 hospital patient-management system inside a Windows XP virtual machine—representing a realistic worst-case modernization scenario—the framework achieved 97.0% mean task success, 7.8 s average execution time, 94.1% OCR field accuracy, and 98.8% workflow recovery across 150 trials, with zero legacy code modification. The self-optimizing delegation mechanism demonstrated autonomous adaptation to systematic execution errors, validating the framework's practical resilience.

LAMA offers healthcare, finance, and government organizations a practical, low-risk modernization pathway that preserves operational continuity while immediately delivering the integration, automation, and accessibility benefits of modern platforms. The framework and evaluation datasets will be released as open-source to support reproducibility and further research in agentic legacy system modernization.

REFERENCES

- [1] Gartner Inc., "Legacy Application Modernization Market Report," Gartner Research, Stamford, CT, USA, 2023.
- [2] K. H. Bennett, "Legacy Systems: Coping with Success," IEEE Software, vol. 12, no. 1, pp. 19–23, 1995.

- [3] M. Lacity and L. Willcocks, *Robotic Process Automation and Risk Mitigation: The Definitive Guide*. SB Publishing, 2017.
- [4] R. Oberhauser and A. Pogolski, "Adaptive Robotic Process Automation," in *Proc. IEEE AIMS*, 2019, pp. 29–36.
- [5] R. Kazman, P. Kruchten, and K. H. Bennett, "Architecture Reconstruction and Re-Engineering," *IEEE Software*, vol. 17, no. 4, pp. 49–57, 2000.
- [6] D. Pérez-Castillo et al., "Mainframe Migration through Screen Scraping: A Case Study," *J. Syst. Softw.*, vol. 136, pp. 64–86, 2018.
- [7] P. Ribeiro, A. Pereira, and H. Santos, "RPA and AI in Organizations: A Systematic Literature Review," *Bus. Process Manag. J.*, vol. 27, no. 6, pp. 1513–1534, 2021.
- [8] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic Process Automation Using Process Mining: A Systematic Literature Review," *Inf. Syst. Front.*, 2023.
- [9] T. Beltramelli, "pix2code: Generating Code from a GUI Screenshot," in *Proc. ACM EICS*, 2018.
- [10] B. Li et al., "RICO: A Mobile App Dataset for Building Data-Driven Design Applications," in *Proc. ACM UIST*, 2017, pp. 845–854.
- [11] J. Luo, H. Xu, and Z. Liu, "OmniParser: A Unified Framework for Visual UI Understanding," 2023, arXiv:2304.00668.
- [12] C. Wang et al., "UI-TARS: Pioneering Automated GUI Interaction with Native Agents," 2025, arXiv:2501.12326.
- [13] R. G. Smith, "The Contract Net Protocol," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, 1980.
- [14] H. P. Nii, "The Blackboard Model of Problem Solving," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986.
- [15] F. Meneguzzi and L. de Silva, "Planning in BDI Agents: A Survey," *The Knowledge Engineering Review*, vol. 30, no. 1, pp. 1–44, 2015.
- [16] A. S. Rao and M. P. Georgeff, "BDI Agents: From Theory to Practice," in *Proc. 1st ICMAS*, 1995, pp. 312–319.
- [17] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in *Proc. WWW*, 2010, pp. 661–670.
- [18] K. Lim, Y. Tan, and M. Lee, "AgentOrchestra: A Hierarchical Multi-Agent Framework," 2025, arXiv:2506.12508.
- [19] R. Gupta and N. Sharma, "DynTaskMAS: A Dynamic Task Graph-Driven Framework," 2025, arXiv:2503.07675.
- [20] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD*, 1987, pp. 249–259.
- [21] B. H. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google, Tech. Rep., 2010.
- [22] M. Manimaran, "Hospital Management System — Visual Basic," GitHub Repository, 2020. [Online]. Available: <https://github.com/manimaran96/HospitalManagementSystemVisualBasic>